

RAG KnowledgeNexus

企业智能文档问答系统

系统架构设计与技术实现文档

文档版本: v1.0

文档日期: 2026-07-05

技术栈: Spring Boot 2.7.18 + Java 17 + MyBatis-Plus + RAG

目录

第一章 项目概览

- 1.1 项目背景与目标
- 1.2 系统定位
- 1.3 核心功能一览

第二章 系统架构

- 2.1 整体架构分层
- 2.2 多模块工程结构
- 2.3 Maven 依赖管理

第三章 技术栈详解

- 3.1 后端核心技术
- 3.2 数据存储层
- 3.3 AI 与向量检索层
- 3.4 基础设施层

第四章 核心执行流程

- 4.1 请求处理链路
- 4.2 文件上传与存储流程
- 4.3 文档解析与分片引擎
- 4.4 向量化与索引流程
- 4.5 RAG 问答执行流程
- 4.6 定时统计任务流程

第五章 模块详解

- 5.1 用户认证模块
- 5.2 知识库管理模块
- 5.3 文档管理模块
- 5.4 智能问答模块
- 5.5 工单系统模块
- 5.6 部门管理模块
- 5.7 数据统计模块
- 5.8 系统日志模块

第六章 配置与部署

- 6.1 多环境配置
- 6.2 关键配置项说明
- 6.3 部署架构

第七章 涉及的开发知识点

- 7.1 Spring Boot 核心知识
- 7.2 Spring Security & JWT
- 7.3 MyBatis-Plus 高级用法
- 7.4 异步与定时任务
- 7.5 对象存储 MinIO
- 7.6 向量数据库 Qdrant
- 7.7 RAG 技术原理
- 7.8 文档解析技术

第一章 项目概览

1.1 项目背景与目标

RAG KnowledgeNexus 是一个基于 RAG (Retrieval-Augmented Generation) 技术架构的企业级智能文档问答系统。旨在解决企业内部海量文档“存得下来、搜得准确、答得智能”三大核心问题。系统允许用户上传 PDF、DOCX、TXT 等格式的文档资料，自动完成文档解析、文本分片、向量化存储，使用者可以通过自然语言提问的方式，由 AI 模型基于知识库内容给出精准答复。

1.2 系统定位

本系统定位为：

- 企业知识库管理平台 — 统一管理企业文档资产
- 智能文档问答助手 — 利用 RAG 技术实现基于知识库的精准问答
- 协同工作平台 — 支持部门管理、权限控制、工单流转
- 数据驱动决策 — 提供使用统计、热点问题分析

1.3 核心功能一览

功能模块	核心能力	涉及主要类
用户认证	登录/注册/JWT签发/Token刷新/头像上传	UserController, UserLoginServiceImpl, JwtUtil
知识库管理	CRUD/可见范围控制/收藏/权限校验	KnowledgeController, KnowledgeServiceImpl
文档管理	上传/预览/分页查询/逻辑删除/重解析	DocumentController, DocumentServiceImpl
文档解析	PDF/DOCX/TXT解析/语义分片/异步处理	DocumentParseServiceImpl, AsyncTaskConfig
向量化	Embedding生成/Qdrant写入/批量处理	VectorizationServiceImpl, EmbeddingServiceImpl
智能问答	RAG检索/LLM生成/对话管理/反馈评分	QaController, QaServiceImpl, ChatServiceImpl
工单管理	工单创建/分页查询/管理员解决	TicketOrderController, TicketOrderServiceImpl
部门管理	树形结构/成员管理/管理员CURD	DepartmentController, DepartmentServiceImpl
数据统计	每日运营数据/热门问题/定时计算	StatController, StatScheduledTask

第二章 系统架构

2.1 整体架构分层

系统采用经典的四层架构设计：

层次	职责	技术选型	所在模块
表现层(Controller)	接收 HTTP 请求、参数校验	Spring MVC / REST API	rag-doc-server
业务层(Service)	核心业务逻辑编排	Spring Bean / @Service	rag-doc-server
数据访问层(Mapper)	数据库 CRUD / 复杂 SQL	MyBatis-Plus / XML	rag-doc-server
基础设施层	JWT、MinIO、OkHttp、线程池	工具类 / 配置类	rag-doc-common/server
AI 能力层	Embedding / LLM 对话	DashScope API / Qdrant	rag-doc-server

请求的典型处理路径为：HTTP Request → Interceptor (JWT) → Controller → Service → Mapper → DB。文档解析和向量化采用异步线程池处理。

2.2 多模块工程结构

项目采用Maven多模块结构：rag-doc-common（公共模块）、rag-doc-model（模型模块）、rag-doc-server（服务模块）。

rag-doc-common（公共模块）：常量定义（DeleteConstants, DocumentParseStatusConstants, UserRoleConstant等）、属性类（MinioProperties, ChatProperties, EmbeddingProperties, QdrantProperties）、工具类（BaseContext ThreadLocal 用户上下文, JwtUtil, MinioUtil）、枚举（EmbedStatusEnum, VisibleScopeEnum）

rag-doc-model（模型模块）：POJO 实体（User, DocDocument, DocChunk, DocKnowledgeBase, Department, QaConversation, QaMessage, TicketOrder等）、DTO（LoginDto, DocumentQueryDto, QaAskDTO, KnowledgeBaseRequest等）、VO（DocDocumentVO, KnowledgeBaseVO, UserListItemVO等）、通用类（ResponseResult, PageResponseResult, AppHttpCodeEnum, BaseEntity）

rag-doc-server（服务模块）：启动类 RagApplication、7个 Controller、Service/Impl 实现类、12个 Mapper、4个配置类、JwtTokenInterceptor、StatScheduledTask 定时任务

2.3 Maven 依赖管理

依赖	版本	作用
spring-boot-starter-parent	2.7.18	Spring Boot 基础框架
spring-boot-starter-web	-	Web MVC & REST API

spring-boot-starter-security	-	BCrypt 密码加密 + 安全配置
mybatis-plus-boot-starter	3.5.3.1	ORM 框架
mysql-connector-j	8.0.33	MySQL 驱动
pagehelper-spring-boot-starter	1.4.6	分页插件
knife4j-spring-boot-starter	3.0.2	API 文档(Swagger)
pdfbox	3.0.2	PDF 解析
poi-ooxml	5.2.5	DOCX 解析
okhttp	4.x	HTTP 客户端(调用AI API)
fastjson2	2.0.49	JSON 处理
jjwt	0.12.x	JWT 令牌
minio	8.x	对象存储客户端

第三章 技术栈详解

3.1 后端核心技术

Spring Boot 2.7.18 自动配置 + 嵌入式 Tomcat。Java 17 文本块等特性。MyBatis-Plus 3.5.3.1 提供便捷 CRUD。Lombok 简化代码。FastJSON2 高性能 JSON。

3.2 数据存储层

MySQL 8.0 存储所有业务数据。MinIO 存储文档原文和头像。

数据库/存储	用途	核心表/桶
MySQL 8.0	结构化数据	sys_user, doc_knowledge_base, doc_document, doc_chunk, qa_conversation, qa_message, sys_dept, ticket_order, stat_daily, sys_operation_log, kb_favorite
MinIO	对象存储	rag-documents(文档), user-avatar(头像)

3.3 AI 与向量检索层

阿里云 DashScope 兼容模式。Embedding: text-embedding-v3 (1024维)。Chat: qwen-plus。Qdrant 向量数据库, Cosine 距离。

每个知识库在 Qdrant 中以独立 Collection 形式存储 (kb_{kbId})。payload 存储 chunk_id, doc_id, chunk_index, content 摘要。支持按 kbId 过滤检索。

3.4 基础设施层

- 异步线程池: core=2, max=4, queue=100, CallerRunsPolicy
- 定时任务: 每天 02:00 执行昨日运营统计
- 日志: 按日滚动, 保留 30 天, 单文件 10MB
- 文件上传: 最大 50MB

第四章 核心执行流程

4.1 请求处理链路

```
Client -> HTTP Request
|
CorsFilter (跨域处理)
|
JwtTokenInterceptor.preHandle
|-- 提取 Authorization: Bearer token
|-- JwtUtil.getClaimsBody() + verifyToken()
|-- 有效 -> BaseContext.setCurrentId(userId)
|-- 无效 -> 401 Unauthorized
|
Controller -> ServiceImpl -> Mapper -> DB
|
ResponseResult 统一封装 -> Client
```

4.2 文件上传与存储流程

1. POST /api/v1/document/uploadDocument (file + kbId)
2. MinioUtil.uploadDocument() -> MinIO rag-documents 桶
3. 创建 DocDocument 实体入库 (parseStatus=PENDING)
4. documentParseService.parseDocumentAsync(document) 异步调用
5. 返回操作成功

4.3 文档解析与分片引擎

```
parseDocumentAsync(document) [Async]
|
1. MinIO 获取输入流
2. 根据 fileType 选择解析器:
PDF -> PDFBox (Loader.loadPDF + PDFTextStripper)
DOCX -> Apache POI (XWPFDocument 段落+表格)
TXT -> UTF-8 读取
3. semanticChunking():
- 按 \n\n 切自然段落
- 段落>500字符按句子(。！？)拆解
- buffer>CHUNK_SIZE(500)时从后找标点切分
- 保留尾部 OVERLAP_SIZE(50)字符作为上下文重叠
4. saveChunks() -> 批量插入 + 异步向量化
5. 更新文档状态 parseStatus=SUCCESS/FAILED
```

4.4 向量化与索引流程

```
vectorizeByDocId(docId) [Async]
|
1. 查询 embedStatus=PENDING 的分片
2. 分批处理(每批25条):
- embedBatch(texts) -> DashScope Embedding API
- 返回 float[1024] 向量
3. qdrantIndexService.upsertVectors():
- ensureCollection(创建Collection, dim=1024, Cosine)
- PUT /collections/kb_{kbId}/points?wait=true
4. 更新分片状态(SUCCESS/FAILED)
```

4.5 RAG 问答执行流程

```
POST /api/v1/qa/ask (conversationId + question)
|
1. embeddingService.embed(question) -> float[1024]
2. qdrantIndexService.search(kbId, vector, topK=5)
- filter: { must: [{ key:kb_id, match:{value} }] }
- 返回 Top-5 相似片段
3. 检索 chunk 内容, 拼接 context
4. chatService.chat(systemPrompt, context+question)
- systemPrompt: 基于文档回答, 不编造
- 调用 DashScope qwen-plus
5. 返回 { messageId, answer, sources }
6. 保存 QaMessage (含 token用量、模型、响应时间)
```

4.6 定时统计任务流程

```
@Scheduled(cron = "0 0 2 * * ?")
StatScheduledTask.computeDailyStats()
|
1. 查询昨日所有 QA -> 计算: totalQa, totalUsers,
positiveRate, avgResponseMs, newDocs
2. 计算热门问题 Top-50 (SHA-256 去重)
3. upsert 到 stat_daily 和 stat_hot_question 表
```

第五章 模块详解

5.1 用户认证模块

管理用户注册、登录、Token 和基本信息。BCrypt 加密密码。JWT 有效期 1 小时，支持刷新。拦截器排除公共路径。

5.2 知识库管理模块

三级可见范围控制：

可见范围	代码	说明
PRIVATE (私有)	0	仅创建者可见
DEPT (部门)	1	本部门可见
PUBLIC (全员)	2	全公司可见

- 列表查询通过 XML Mapper 拼接权限条件
- 管理员不受限制
- 收藏使用 kb_favorite 表，支持切换

5.3 文档管理模块

- 上传 -> MinIO -> DB -> 异步解析
- 列表分页 + 多种条件筛选
- 预览使用 MinIO 预签名 URL
- 仅作者可更新/删除
- 逻辑删除 (is_deleted 标记)
- 重解析：删旧分片 -> 重新解析 -> 重新向量化

5.4 智能问答模块

核心 RAG 业务模块。对话管理（创建/列表/详情/追问）。RAG 组件：EmbeddingServiceImpl（向量化）、QdrantIndexServiceImpl（检索）、ChatServiceImpl（LLM生成）。质量评分反馈。

5.5 工单系统模块

- 创建工单（标题、描述、优先级、关联消息）
- 列表：员工看自己的，管理员看全部
- 解决：管理员填写回复
- 状态：待处理/处理中/已解决/已关闭

5.6 部门管理模块

- 基于 parentId 构建无限级树
- 平铺列表按 sort 排序
- 部门成员查询
- 管理员 CRUD，有子部门或成员时禁止删除

5.7 数据统计模块

- 每日统计：总问答量、活跃用户、好评率、转工单数、响应时间、新增文档数
- 热门问题：归一化+Hash 聚合 Top-50
- 定时计算：每天 02:00
- BigDecimal 计算百分比

5.8 系统日志模块

- SLF4J + Logback，按日滚动
- ROOT=INFO, com.rag.system=DEBUG
- 保留 30 天，单文件 10MB

第六章 配置与部署

6.1 多环境配置

Spring Profile 机制：dev（默认）和 prod（环境变量注入）。

配置	开发环境 (dev)	生产环境 (prod)
MySQL	192.168.150.101:3306	mysql-prod:3306(变量)
MinIO	192.168.150.101:9000	环境变量注入
日志级别	DEBUG	WARN/INFO
日志路径	logs/app.log	/var/log/rag-system/app.log

6.2 关键配置项

配置项	示例值	说明
server.port	8085	服务端口
max-file-size	50MB	单文件大小
rag.minio.endpoint	http://192.168.150.101:9000	MinIO 地址
rag.embedding.model	text-embedding-v3	向量模型
rag.chat.model	qwen-plus	对话模型
rag.qdrant.dimension	1024	向量维度

6.3 部署架构

```
Nginx / 负载均衡（可选）
|
RAG KnowledgeNexus 服务（Spring Boot, port 8085）
|--- MySQL 8.0（主数据）
|--- MinIO（对象存储）
|--- Qdrant（向量数据库）
|--- DashScope AI API（外部服务）
```

第七章 涉及的开发知识点

7.1 Spring Boot 核心知识

- 自动配置: @EnableAutoConfiguration
- IoC 容器: @Component, @Service, @Controller
- 属性绑定: @ConfigurationProperties
- 消息转换器: MappingJackson2HttpMessageConverter
- CORS: CorsFilter 跨域配置
- 文件上传: MultipartFile

7.2 Spring Security & JWT

- BCryptPasswordEncoder 密码加密
- CSRF 关闭, Token 认证代替 Session
- JWT 结构: Header.Payload.Signature, HS256
- JWT 库: Jwts.builder() / parser()
- Token 刷新策略: 1小时有效期, 5分钟缓冲
- HandlerInterceptor 拦截器模式

7.3 MyBatis-Plus 高级用法

- BaseMapper 内置 CRUD
- LambdaQueryWrapper 类型安全条件
- 自定义 XML Mapper 多表关联
- PaginationInnerInterceptor + Page
- 雪花算法 @TableId(type=ASSIGN_ID)
- 手动逻辑删除 is_deleted
- saveBatch 批量插入

7.4 异步与定时任务

- @EnableAsync + @Async("executorName")
- ThreadPoolTaskExecutor 自定义线程池
- CallerRunsPolicy 拒绝策略
- @EnableScheduling + @Scheduled(cron)

7.5 对象存储 MinIO

- MinioClient 客户端
- PutObjectArgs / GetObjectArgs

- 预签名 URL (GetPresignedObjectUrlArgs)
- 多桶隔离 (文档/头像)
- UUID 文件名防冲突

7.6 向量数据库 Qdrant

- Collection 管理 (每知识库独立)
- REST API: PUT points / POST search
- Payload 过滤 filter.must
- Cosine 距离算法
- OkHttp 直接调用 Qdrant API

7.7 RAG 技术原理

- RAG = Retrieval + Augmented + Generation
- Indexing: 文档 -> 分片 -> Embedding -> 索引
- Retrieval: 问题 -> Embedding -> 相似度检索
- Generation: 问题 + 上下文 -> LLM -> 回答
- 优势: 解决知识截止、减少幻觉、支持私有知识
- 语义分片: 自然段落+句子边界+Overlap
- System Prompt 约束 LLM 行为

7.8 文档解析技术

- PDFBox 3.0: Loader.loadPDF + PDFTextStripper
- Apache POI 5.2.5: XWPFDocument 段落+表格
- TXT: Java NIO UTF-8 读取
- 状态机: PENDING->PARSING->SUCCESS/FAILED

文档信息

文档名称: RAG KnowledgeNexus 系统架构与技术文档

文档版本: v1.0

生成日期: 2026-07-05

技术栈: Spring Boot 2.7.18 | Java 17 | MyBatis-Plus | MySQL 8.0 | MinIO | Qdrant | DashScope